UFUG 2602
递归

Beiyan Cao, Tengfei Chang

CES and IoT Thrusts
香港科技大学（广州）

大纲

- 递归
  - **引言**
    - 现实世界示例
    - 定义
  - 示例
    - 二分查找
    - 斐波那契数列

现实世界示例：俄罗斯套娃



- 俄罗斯套娃是一组玩具，每个娃娃里套着更小的娃娃，直到最小的那个。

- 打开第一个套娃：

1. 若里面还有套娃，继续打开。
2. 若里面没有套娃，则停止。



https://www.youtube.com/watch?v=93_iFq6rBy8

引言

- 维基百科对递归的定义：
  - 递归是一种求解计算问题的方法，其解依赖于同一问题更小实例的解。
  - 递归通过函数在自身代码中调用自己来求解这类递归问题。
- 处理子问题时，可将其视为已解决，并直接使用子问题的输出继续设计算法。

- 考虑下列方程
  - $f(n) = n * f(n-1)$, if $n \geqslant 1$
  - $f(n) = 1$, if $n = 0$
- f(3) 的值是多少?

将 f() 实现为递归函数

```python
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

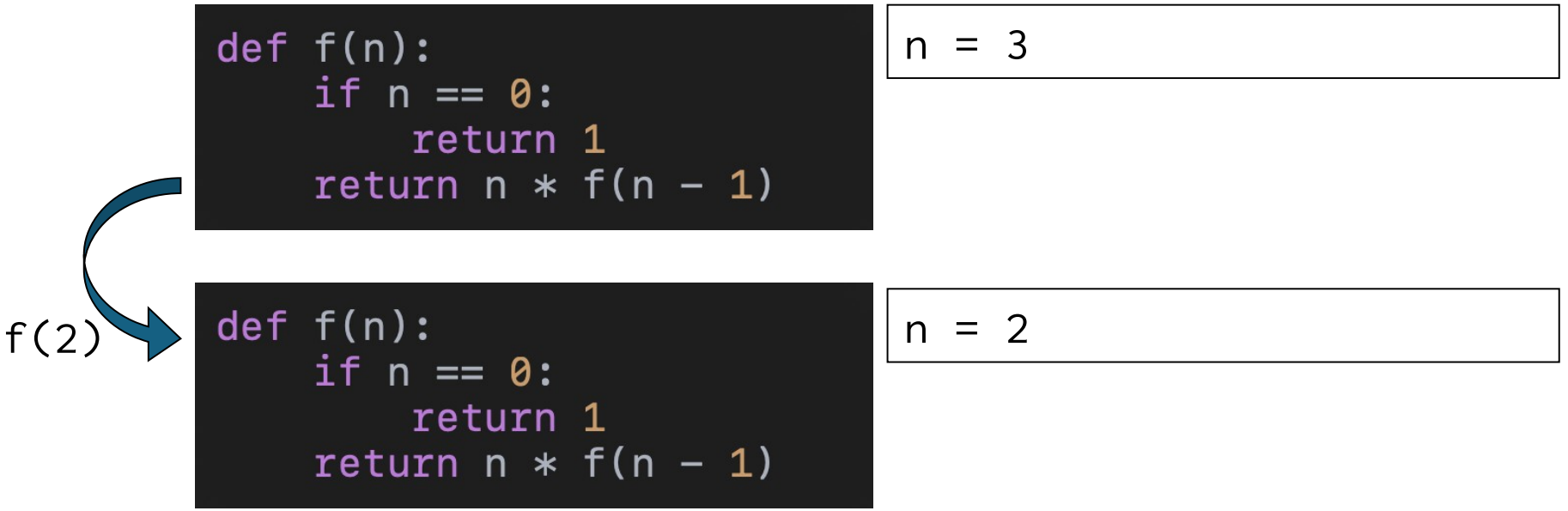
在算法设计中，可以直接假定 f(n-1) 已求解。
但当函数调用“自身”时会发生什么？

f(3)

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 3

最初， main() 调用 f(3) 。
在这次调用中， n 由 main() 设为 3 。



从概念上说，函数并非真正调用“自身”，而是调用自身的一个“克隆”。

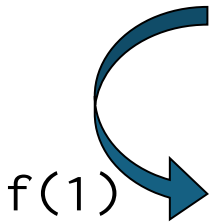
每个函数“克隆”都有自己的局部变量。

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 3

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 2




```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 1

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

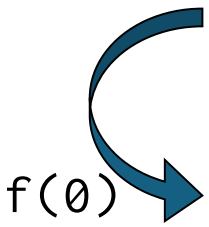
n = 3

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 2

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 1




```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 0

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 3

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 2

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 1

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 0



返回 1

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

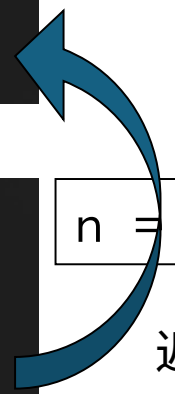
n = 3

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 2

```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 1



返回
1 * f(0)
= 1 * 1
= 1

```python
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 3

```python
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 2

返回
2 * f(1)
= 2 * 1
= 2

```python
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

n = 3

返回
3 * f(2)
= 3 * 2
= 6

最终返回 6 。

引言

- 递归函数有两个主要组成部分：
- **基本情况（终止情况）：**
  - 函数直接产生结果（不再递归）并返回。

- 当递归无法终止时，计算机最终会耗尽内存中的“栈空间”，并产生名为“栈溢出”（stack overflow）的运行时错误。

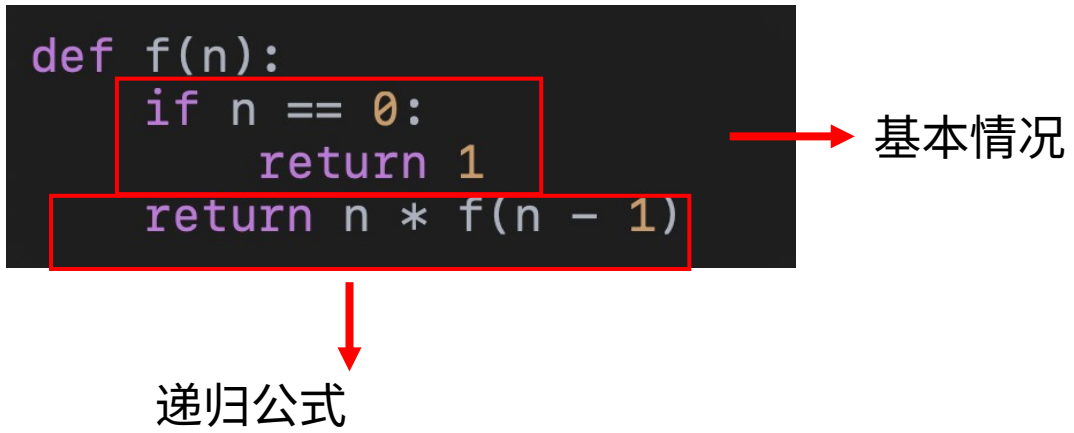
```
def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)
```

基本情况

引言

- 递归函数有两个主要组成部分：
- **基本情况（终止情况）：**
  - 函数直接产生结果（不再递归）并返回。

- **递归公式：**
  - 将当前问题分解为更小的子问题。



练习 1 — 偶数求和

- 定义一个递归函数 **sumEven(n)**，用于计算
  - 2+4+6+…+2n

- 基本情况是什么?
- 递推公式是什么?
- 计算 sumEven(5)。

练习 2 - 统计出现次数

- 编写递归函数 **countChar(s, c)**，统计字符 c 在字符串 s 中出现的次数。

- 基本情况是什么?
- 递推公式是什么?
- 计算 countChar("BANANA", "A")。

练习 3 - 回文判断

- 编写递归函数 **isPalindrome(s)** 。回文是指正读和反读都相同的字符串。

- 基本情况是什么?
- 递推公式是什么?
- 求值:
  - isPalindrome("LEVEL")
  - isPalindrome("DATA")

练习 4 - 递归数组求和

- 设 **arraySum(arr, n)** 返回数组前 n 个元素之和。

- 基本情况是什么?
- 递推公式是什么?
- 计算 arraySum([4, 7, 2, 9, 3], 5) 。

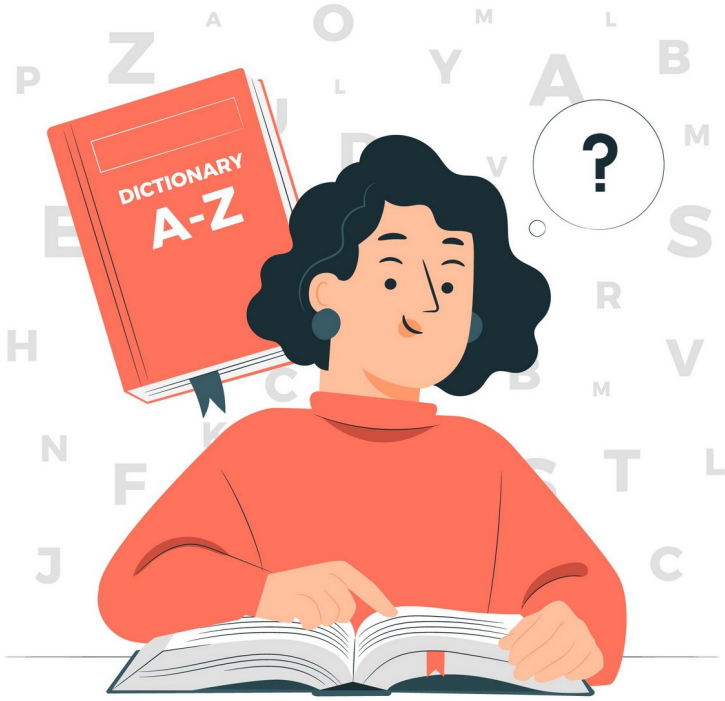
大纲

- 递归
  - 引言
    - 现实世界示例
    - 定义
  - **示例**
    - 二分查找
    - 斐波那契数列

二分查找——示例

在词典中查找单词
在词典中查找单词时，不会从第一页开始逐个查找。而是：

1. 打开词典靠近中间的位置，判断目标单词在当前页之前还是之后。
2. 据此，在词典的前半部分或后半部分继续查找。
3. 该过程持续进行，不断将查找范围减半，直到找到该单词。



二分查找

- **思路**

1. 从已排序数组开始。

2. 找到中间元素：
   若中间元素等于目标值，返回其位置。
   若中间元素大于目标值，在左半部分查找。
   若中间元素小于目标值，在右半部分查找

3. 重复步骤 2 ，直到找到目标或查找区间为空。

二分查找：迭代版本

- 假设整数集合已排序。
- 将 search_num 与位于中间的元素（第 $(n/2)$ 个，来自 S）进行比较）

```
def binary_search(sorted_set, search_num):
    left = 0
    right = len(sorted_set) - 1

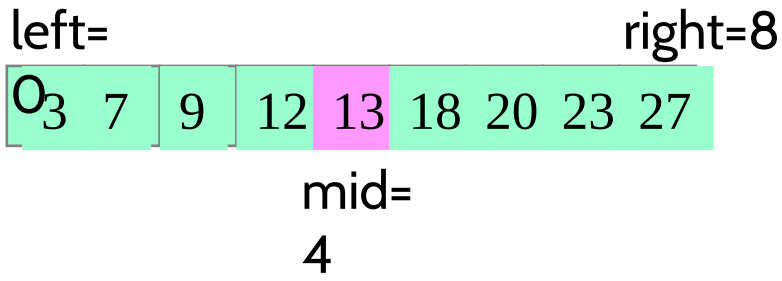
    while left <= right:
        middle = (left + right) // 2  # Find the middle index

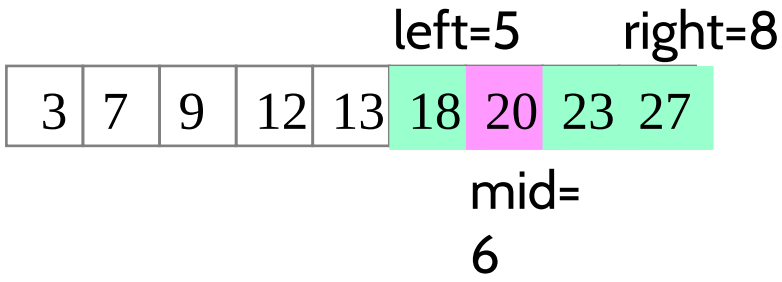
        if sorted_set[middle] < search_num:
            left = middle + 1  # Narrow search to the right half
        elif sorted_set[middle] > search_num:
            right = middle - 1  # Narrow search to the left half
        else:
            return middle  # Return the index if found

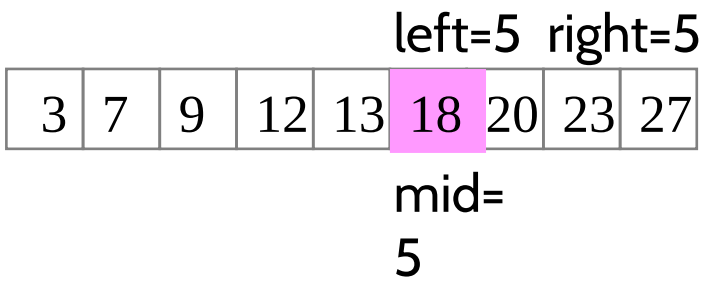
    return -1  # Return -1 if not found
```

searchnum = 18 size = 9

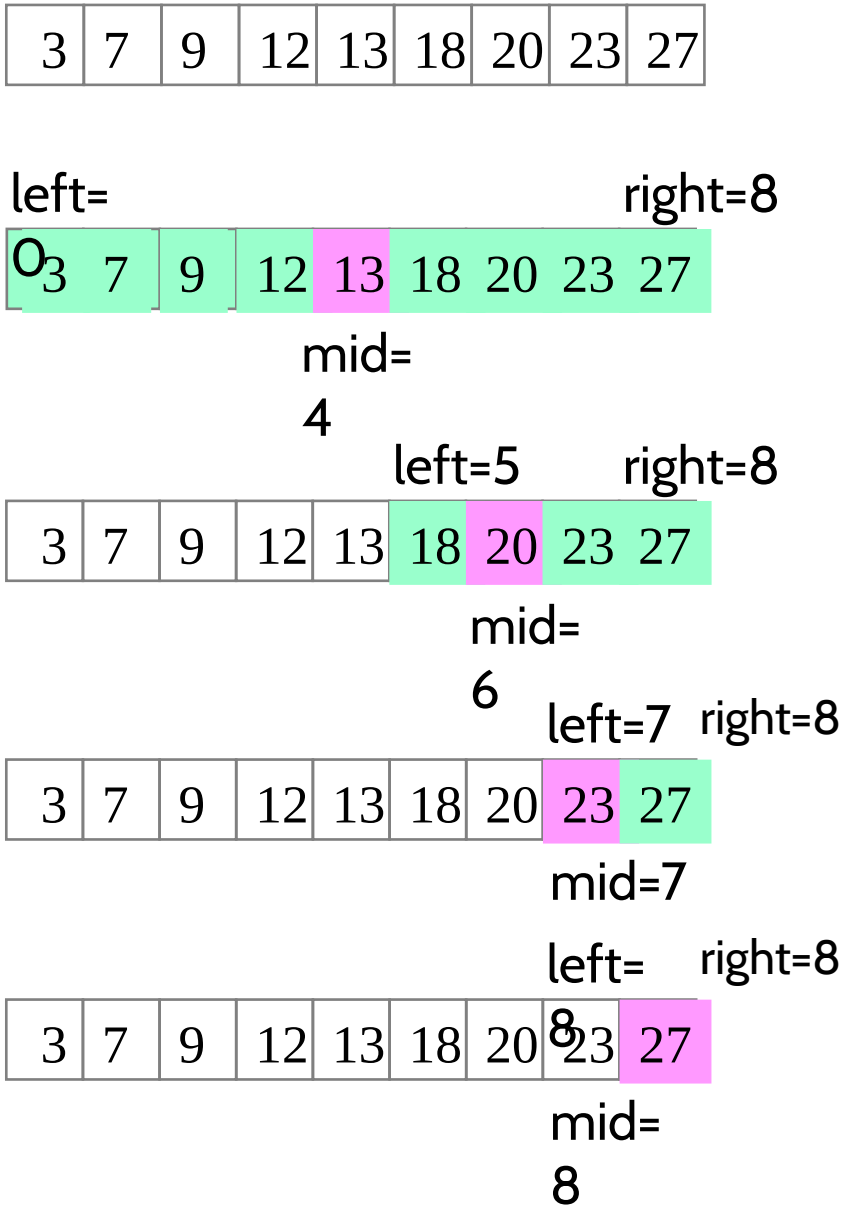








searchnum = 28 size = 9



递归与迭代的比较

	递归	迭代
重复	通过调用自身	通过使用迭代结构（for、while）
终止	基本情况	终止条件

大多数情况下，迭代与递归可以相互转换。

二分查找：递归版本

算法： Binary Search(arr, searchnum, left, right)

1	if left = right	$O(1)$
2	if arr[left]= searchnum	$O(1)$
3	return left	$O(1)$
4	else	
5	return -1	$O(1)$
6	middle ←(left + right)/2	$O(1)$
7		
8		
9		
10		
11		
12		

问题：递归公式是什么？

二分查找：递归版本

算法： Binary Search(arr, searchnum, left, right)

1	if left = right	$O(1)$
2	if arr[left]= searchnum	$O(1)$
3	return left	$O(1)$
4	else	
5	return -1	$O(1)$
6	middle ←(left + right)/2	$O(1)$
7	if arr[middle] = searchnum	$O(1)$
8	return middle	$O(1)$
9	elseif arr[middle] < searchnum	$O(1)$
10	return BinarySearch(arr, searchnum, middle+1, right)	$O(?)$
11	else	
12	return BinarySearch(arr, searchnum, left, middle -1)	$O(?)$

二分查找：递归版本

- 给定输入规模 n，令 $g(n)$ 为二分查找中执行的基本操作总数

算法： Binary Search(arr, searchnum, left, right)

```
1   if left = right
2       if arr[left]= searchnum
3           return left
4       else
5           return -1
6   middle ←(left + right)/2
7   if arr[middle] = searchnum
8           return middle
9   elseif arr[middle] < searchnum
10      return BinarySearch(arr, searchnum, middle+1, right)
11      else
12      return BinarySearch(arr, searchnum, left, middle-1)
```

我们仍可统计该部分的基本操作次数

执行的基本操作总数是一个与输入规模 n 无关的常数，我们可以用 a 来表示

要么执行第 10 行，要么执行第 12 行，但不会两者都执行。第 10 行或第 12 行执行的基本操作次数是多少?

递归二分查找的分析 (i)

- 假设 n 是 2 的幂。
- 对于 $n = 1$，显然
  - $g(n) = b$，其中 b 为某个常数。
- $g(n)$ 也可以递归定义：
  - 执行 a 次基本操作后，将输入规模减半；
  - 然后，对规模至多为 $n/2$ 的输入执行递归二分查找；
    - 根据定义，该子问题的运行时间至多为 $g(n/2)$。



$$g(n) \leq a + g\left(\frac{n}{2}\right)$$

递归二分查找的分析（ii）

- 已知 $g(n)=a+g(n/2)$，$g(1)=b$
  - 如何用 a 和 b 表示 $g(4)$？

$$
\begin{aligned}
g(4) &\le g(2)+a \\
&\le g(1)+a+a \\
&\le 2a+b
\end{aligned}
$$

  - 若 $n=2^x$，如何用 a 和 b 表示 $g(n)$？

$$
\begin{aligned}
g(n) &\le g\left(\frac{n}{2}\right)+a \\
&\le g\left(\frac{n}{2^2}\right)+a+a \\
&\le g\left(\frac{n}{2^3}\right)+a+a+a \\
&\le g\left(\frac{n}{2^4}\right)+a+a+a+a \\
&\le g(1)+\underbrace{a+a+\cdots+a+a}_{\text{共 } \log_2 n \text{ 个}} = a\cdot\log_2 n+b = O(\log_2 n)
\end{aligned}
$$

递归二分查找的分析（iii)

如何处理一般的、不是 2 的幂的 n？

- 可以采用取整方法。令 $n^{\wedge\prime}$ 为大于 n 的最小的 2 的幂。于是有 $n'<2n$。
- 于是有：
- $f(n)\le f(n')$
- $\le a\cdot\log_2 n'+b$ （先前已证）
- $< a\cdot\log_2(2n)+b$
- $= a\cdot(1+\log_2 n)+b$
- $= a\cdot\log_2 n+a+b = O(\log n).$

练习 5 — 每次减 2

```
def mystery(n):
    if n <= 1:
        return 1
    return n + mystery(n - 2)
```

- 写出 $T(n)$ 的递推关系式。
- 递归共有多少层?
- 时间复杂度是多少?

练习 6 — 反复减半

```
def mystery(n):
    if n <= 1:
        return 1
    return mystery(n // 2) + n
```

- 写出 $T(n)$ 的递推关系式。
- 每次递归调用时，输入规模如何变化?
- 时间复杂度是多少?

练习 7 — 两个减半子问题 + 线性工作量

```
def mystery(n):
    if n <= 1:
        return 1

    for i in range(n):
        print(i)

    mystery(n // 2)
    mystery(n // 2)
```

- 写出递推关系式： $T(n)=?$
- 递归树每一层的工作量是多少?
- 总时间复杂度是多少?

练习 8 — 三个更小的子问题

```
def mystery(n):
    if n <= 1:
        return 1

    mystery(n // 2)
    mystery(n // 2)
    mystery(n // 2)
```

- 写出 $T(n)$ 的递推关系式。
- 递归树中每个结点有多少个子结点?
- 递归树的深度是多少?
- 时间复杂度是多少?

示例：斐波那契数

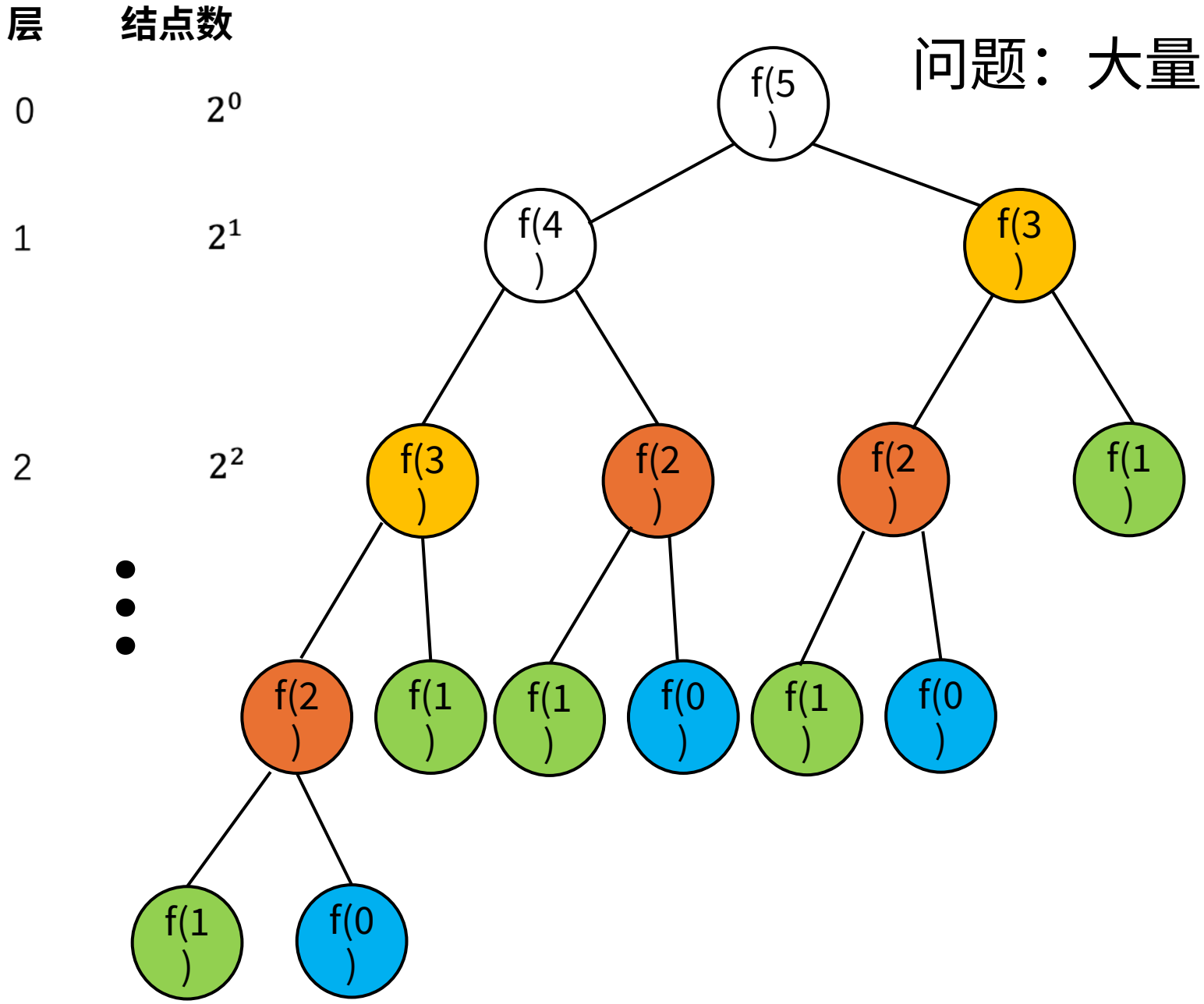
- 斐波那契数是一个数列：
  - 数列中的第 n 项取决于同一数列中的前两项。
  - 例如： 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, …

$$f(x) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

第 n 个斐波那契数： f(n)

```
if n is 0:  return 0;
if n is 1:  return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```

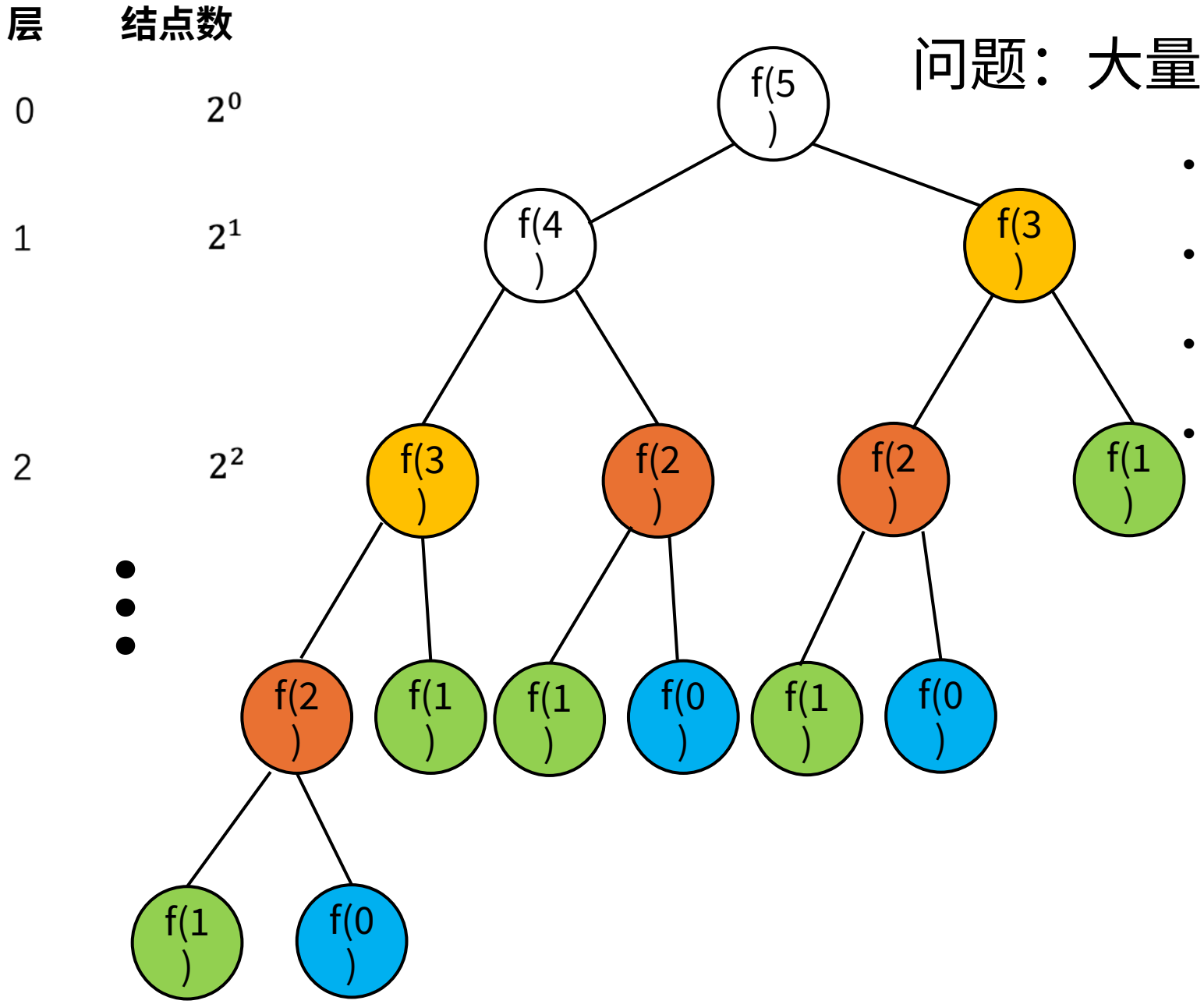
问题：大量重复计算



第 n 个斐波那契数：f(n)

```
if n is 0:  return 0;
if n is 1:  return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```

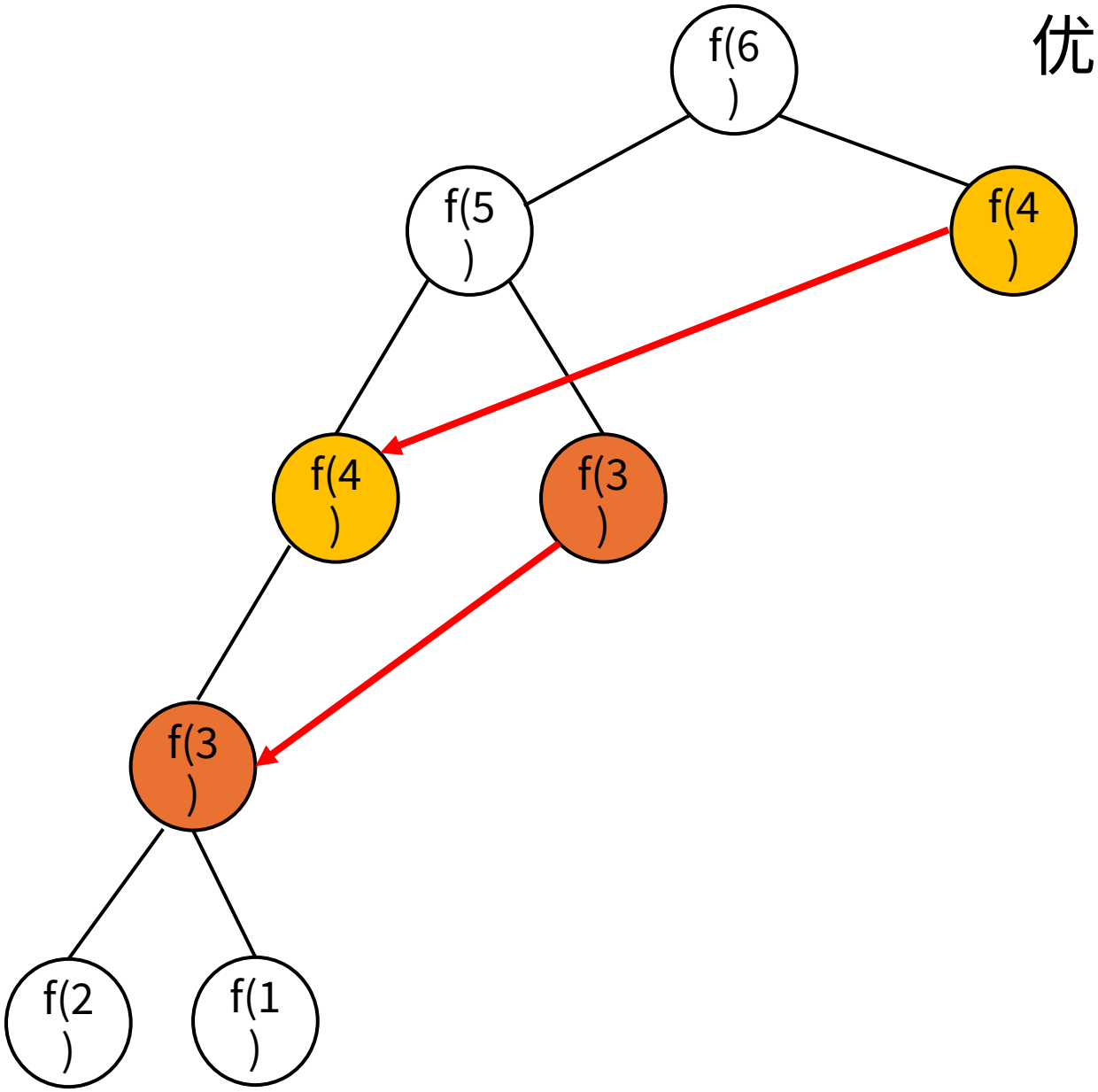
问题：大量重复计算



- 每个结点对应一次函数调用。
- 在大约前 $n/2$ 层，结点数逐层翻倍。
- 到第 $n/2$ 层时，结点数量已经是指数级的。
- 因此运行时间是指数级的。

第 n 个斐波那契数：f(n)

```
if n is 0:  return 0;
if n is 1:  return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```



优化方法：记忆化

- 用列表存储中间结果 f(i)。
- 通过只计算一次 f(i) 来减少重复计算。
- 每层最多有 2 个结点；结点总数为 $O(n)$。
- 时间复杂度为 $O(n)$。

第 n 个斐波那契数：f(n, memo = [])

```
if n is 0:  return 0;
if n is 1:  return 1;
if n not in memo:
    memo[n]= f(n-1,memo) + f(n-2,memo)
return memo[n]
```

练习 9 — 设计你自己的递归函数

- 设计一个递归函数：

```
is_sorted(arr, n)
```

- 若 arr 的前 n 个元素按**非递减顺序**排列，则返回 True。
- 示例：

```
is_sorted([2, 4, 4, 7, 9], 5) → True
is_sorted([2, 5, 3, 7], 4)    → False
```

要求：

- 定义**基本情况**。
- 定义**递推公式**。
- 实现该递归函数。
- 它的**时间复杂度**是多少？